

LECTURE

pyspeller

A P300 speller system: software architecture and signal processing

Implementation: 4,550 lines of Python, 145 tests, numpy only

github.com/berdakh/pyspeller

berdakh.github.io/pyspeller



Background

1 A rare event you are attending to
produces a positive deflection in the EEG about 300 ms later, called the P300.

2 The grid flashes rows and columns
Two of the twelve flashes contain the attended letter; the other ten do not.

3 The letter is the intersection
The row and the column with the largest response identify one symbol.



Target flashes (blue) against non-target flashes (grey), averaged over a session.

Note

A single flash gives a very unreliable measurement. The rest of the system is concerned with making the decision reliable.

Outline

1

System architecture

Design requirements, the buffer, the protocol, timing

2

System components

Acquisition, stimulus presentation, phase control

3

Signal processing

Epochs, preprocessing, classification, decoding

4

Implementation notes

Data storage, user interface, testing

5

Demonstration and extensions

A recorded run, and possible project work

Learning objectives

After this lecture you should be able to:

1. describe the client-server structure;
2. explain how events and EEG samples are aligned in time;
3. follow the path from samples to a letter;
4. modify and test part of the system.

Design requirements

1 Accurate timing

A 5 μ V response 300 ms after a flash can only be measured if the time of the flash is known to within a few milliseconds.

2 Concurrent tasks

Stimulus presentation, data acquisition, classification and the user interface must run without blocking each other.

3 Hardware independence

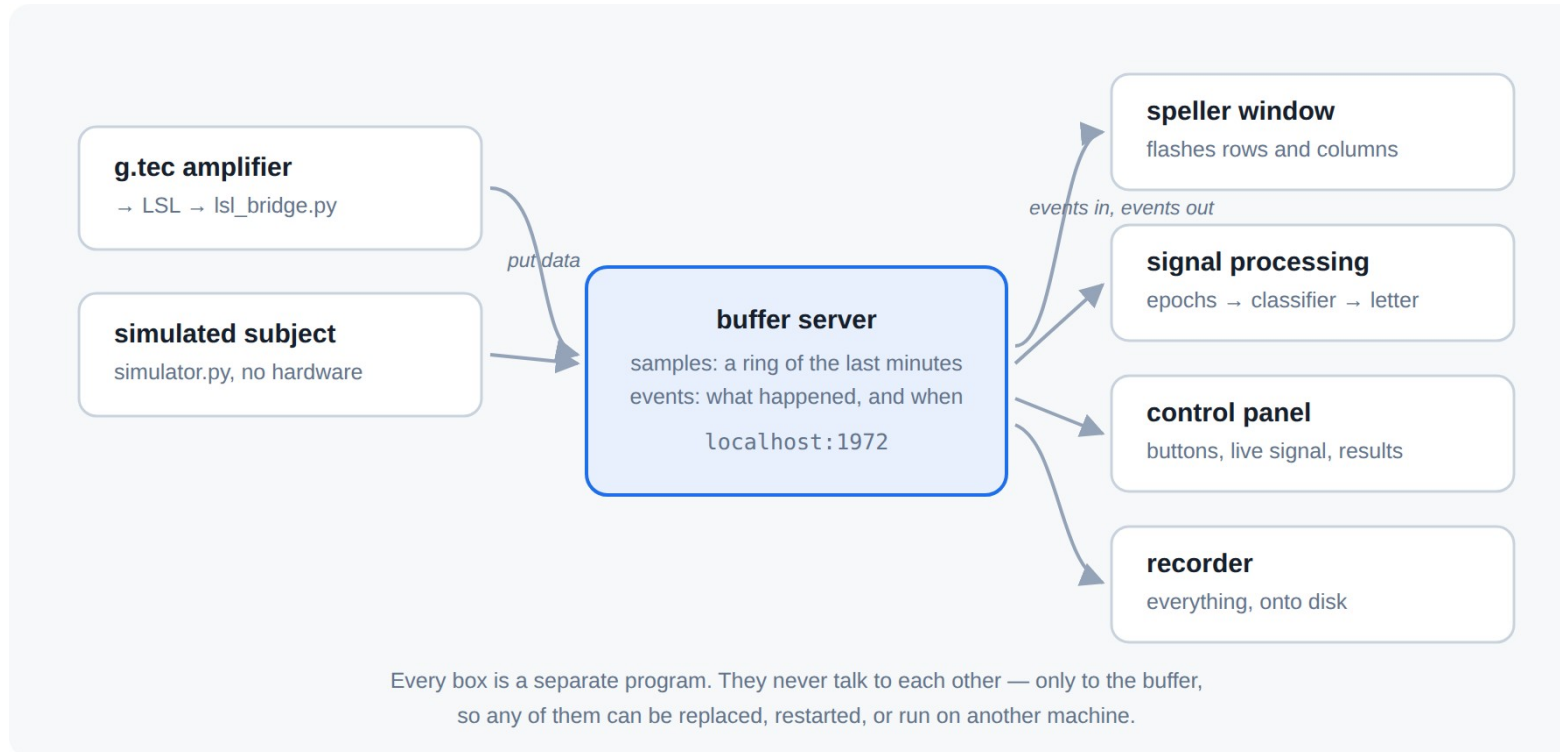
The same experiment must run with a g.tec amplifier, another device, a recording or a simulator.

4 Offline reproducibility

A recorded session must be re-analysable later, without the participant and without the hardware.

One program can satisfy two or three of these; the architecture that follows satisfies all four.

Architecture: buffer and clients



The FieldTrip buffer: a TCP server holding a ring of recent samples and a list of events. All other programs are clients of it.

Design rules

- Samples are appended, never modified
- Events are timestamped by sample index
- Any client may read any range at any time
- Clients communicate only through the buffer

Advantages and costs of this design

1 Interchangeable data sources

Simulator, LSL device or recorded file: the experiment code is unchanged.

2 Language independence

The MATLAB buffer_bci client and this Python one speak the same protocol.

3 Record and replay

Every sample and event is already in one place, in order.

4 Fault isolation

If the classifier fails, stimulus presentation continues; the client can be restarted.

5 Several readers, one stream

The interface, the analysis and the recorder read the same data concurrently.

Costs

Four processes must be started instead of one.

The data exist in a second copy in memory.

The protocol has to be implemented and maintained.

These costs are usually accepted when the hardware changes, when clients may fail, or when sessions must be recoverable.

The buffer protocol

```
request:  [version | command | size] + payload  
          uint16   uint16   uint32
```

PUT_HDR	0x101	declare channels + rate
PUT_DAT	0x102	append samples
PUT_EVT	0x103	append an event
GET_DAT	0x202	read a sample range
WAIT_DAT	0x402	block until new data arrives

Summary

An eight-byte header, a payload and a reply. In this implementation the client is about 300 lines and the server about 250.

Blocking reads: WAIT_DAT

- A client requests: return when sample 4 096 exists, or after 500 ms, whichever occurs first.
- This avoids polling loops and fixed sleep intervals.
- It is how the signal processor stays synchronised with an amplifier it never addresses directly.
- The stimulus client uses the same call to wait for a prediction.

Timing: events carry sample indices

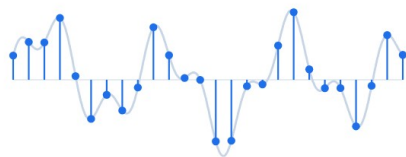
from a wave to numbers

1 · a voltage that never stops changing



What comes out of the amplifier is a smooth voltage, changing all the time.

2 · measured, over and over



512 times a second the converter reads the voltage and writes down a number.

3 · a stream of numbers

2.41 μV
-3.88 μV
7.02 μV
11.64 μV
4.19 μV
-6.50 μV
-12.07 μV

→ to LSL

24 bits — about 16 million steps — so a single microvolt is still several steps.

An event is timestamped with the number of samples already stored when it arrived, not with a clock reading.

Extracting the epoch after a flash is therefore array indexing, samples n to $n+77$, and not a calculation involving clocks.

```
# 128 Hz, 600 ms epochs
fsample    = 128
trlen_ms   = 600
trlen      = 77 samples
```

```
event: stimulus.rowFlash
      value  = 3
      sample = 4096
```

```
epoch = data[4096:4173]
      shape (8, 77)
```

```
# no clocks were consulted
```

Software components

buffer/	protocol, server, client	<i>protocol implementation</i>	~750
acquisition/	simulator, lsl_bridge, saver	<i>data sources and recording</i>	~700
speller/	matrix, stimulus, render, sigproc	<i>the experiment</i>	~1.1k
signalproc/	preproc, epochs, classifier	<i>numerical methods</i>	~600
gui/	launcher, control_panel, training_view	<i>user interface only</i>	~800

4.5k

lines of Python

145

tests

5

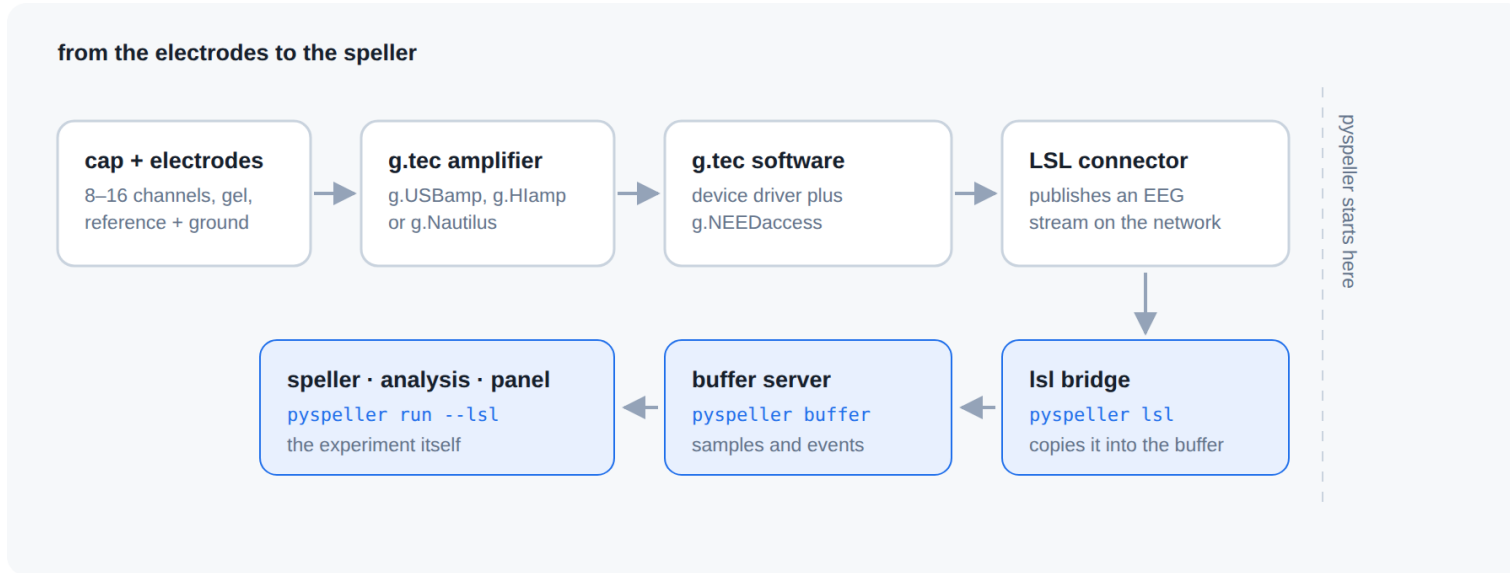
packages

3

dependencies

numpy; tkinter for the interface and pylsl for an amplifier connection.

Data acquisition



Signal path with a g.tec amplifier: cap, amplifier, vendor driver, LSL, bridge, buffer.

All three use the same PUT_HDR and PUT_DAT calls, so the remaining components are independent of the data source.

Three sources

Simulator

Generates EEG with an event-related response, mains interference, blinks and drift. Deterministic, no hardware required.

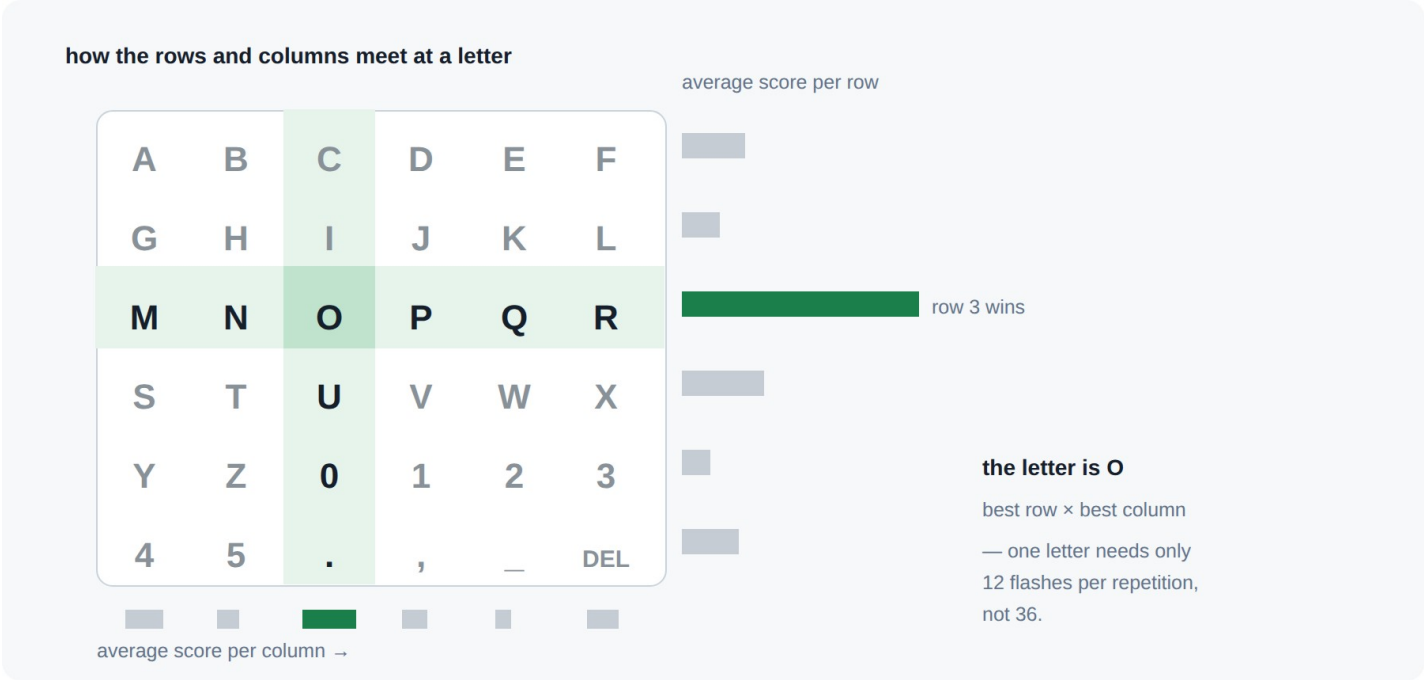
LSL bridge

Subscribes to a Lab Streaming Layer EEG stream and copies samples into the buffer. This is the path used with g.tec amplifiers.

Recorded session

Replays a stored recording sample by sample at the original rate.

Stimulus presentation



Twelve groups cover thirty-six symbols: each symbol is the intersection of one row and one column.

12

groups (6 rows + 6 cols)

12

repetitions per letter

144

flashes per letter

150 ms

between flash onsets

100 ms

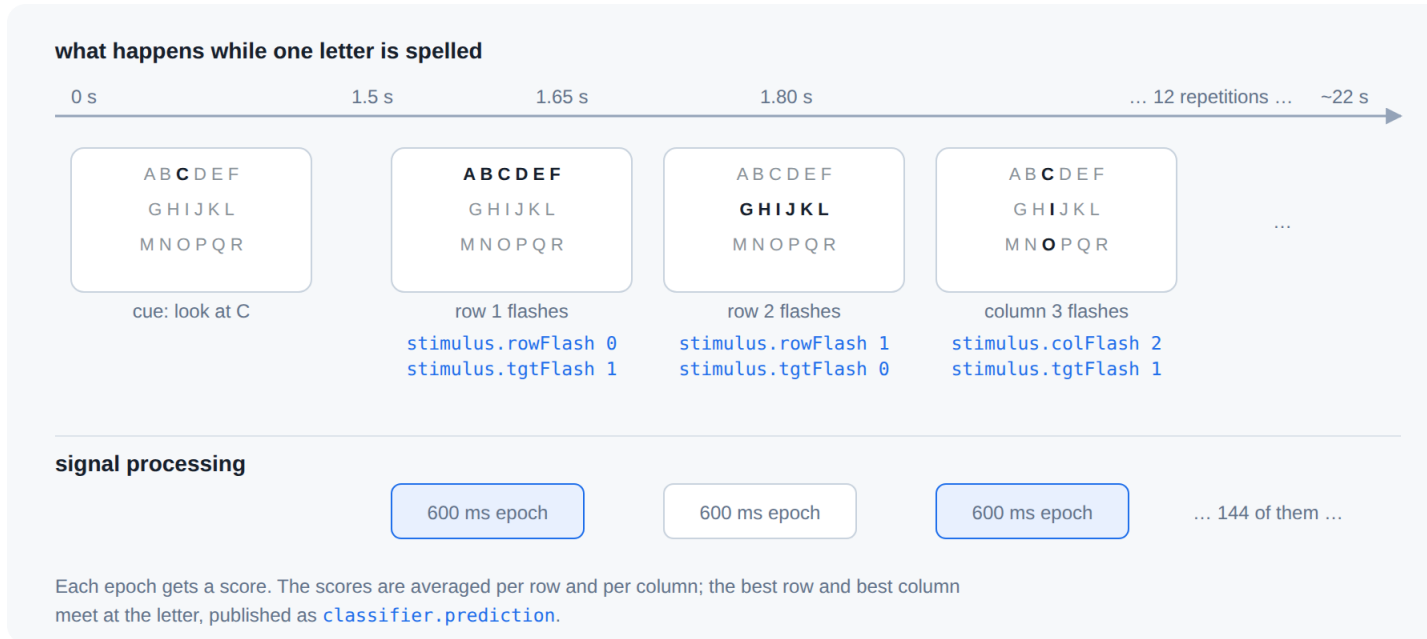
a group stays lit

≈ 22 s

to spell one letter

Note: the order is randomised, with at least three flashes between repetitions of one group.

Experiment phases and control events



```
startPhase.cmd
  calibrate | train | feedback
```

```
speller.ready
  "I have the command"
```

```
stimulus.rowFlash 3
stimulus.colFlash 5
stimulus.tgtFlash 1
  the training label
```

```
classifier.prediction R
speller.control pause
```

A session is a sequence of phases, each announced as an event.

The clients share no memory and call no functions in each other. The list above is the complete interface between them.

Epoch extraction

1 Wait for a flash event

Block until the event exists, using WAIT_DAT.

2 Wait 600 ms more

The epoch is not complete until the samples after the flash have arrived.

3 Slice

`data[n : n+77]` → an 8×77 array.

4 Attach the label

`stimulus.tgtFlash` gives 1 for a target flash and 0 otherwise.

One calibration word, five letters

5 letters x 144 flashes = 720 epochs
120 targets · 600 non-targets
imbalance 1 : 5 - by construction

Every epoch: 8 channels × 77 samples

Class imbalance

A classifier that always predicts "non-target" is correct 83 % of the time and cannot spell. Performance is therefore reported as AUC rather than accuracy.

Preprocessing

1 Detrend

Fit and subtract a straight line from each epoch, removing slow drift.

2 Common average reference

Subtract the mean over channels, removing components common to all electrodes: interference, distant muscle activity, reference drift.

3 Band-pass 0.5-10 Hz

Trapezoidal filter applied in the frequency domain, giving zero phase distortion of the response.

4 Downsample 128 → 16 Hz

The response contains no detail above 10 Hz; 77 samples per epoch become 10.

5 Reject outliers

Remove channels and epochs whose power differs strongly from the rest: blinks, electrode pops, poor contact.

Shape bookkeeping

in	(8, 77)
filter	(8, 77)
resample	(8, 10)
flatten	80

80 features

720 examples

Nine training examples per feature.
This ratio is the reason for using a regularised linear model.

Classification: linear discriminant analysis

```
mu1, mu0 = class means
S         = pooled covariance
S         = (1-l)*S + l*trace(S)/d*I
w         = inv(S) @ (mu1 - mu0)

score(x) = w @ x + b
```

Linear model

With 80 features and 720 examples, and a response that is an additive voltage difference, non-linear models overfit.

Shrinkage

An 80×80 covariance matrix estimated from 720 epochs is poorly conditioned. It is shrunk toward a diagonal matrix; here $\lambda = 0.6$.

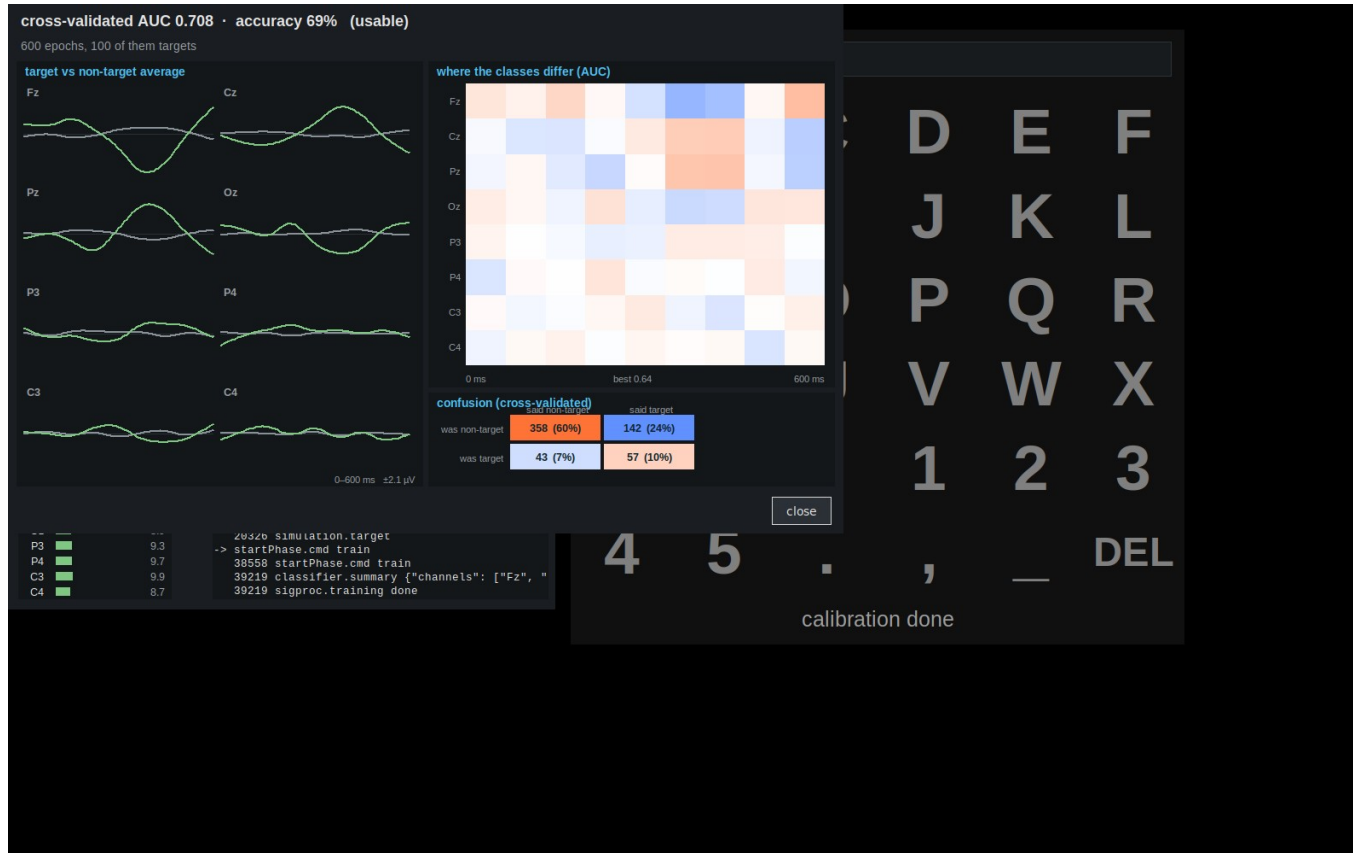
Implementation

Written directly in numpy (about forty lines) so that every step can be read and modified. Training takes milliseconds.

Interpretation of the score

- One value per flash, expressing how target-like that epoch is.
- It is not a probability; only the relative ordering of the values is used.
- A single value is only slightly better than chance, as expected.
- A decision uses 24 values: 12 repetitions of the two groups containing the symbol.

Evaluating the classifier



AUC

Probability that a randomly chosen target epoch scores higher than a randomly chosen non-target epoch.

ERP panel

Whether the target and non-target averages separate around 300 ms.

Discriminability map

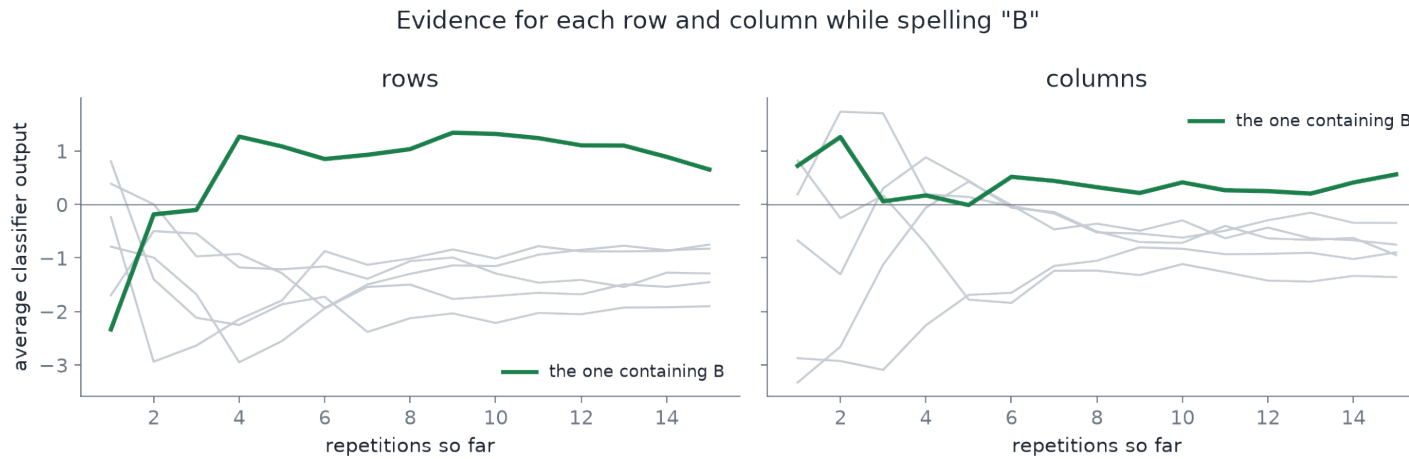
Which channels and time points carry the discriminating information.

Confusion matrix

Computed by five-fold stratified cross-validation, not on the training data.

0.50 chance · 0.60-0.75 usable
0.75-0.85 good · 0.85+ excellent

Evidence accumulation and decoding



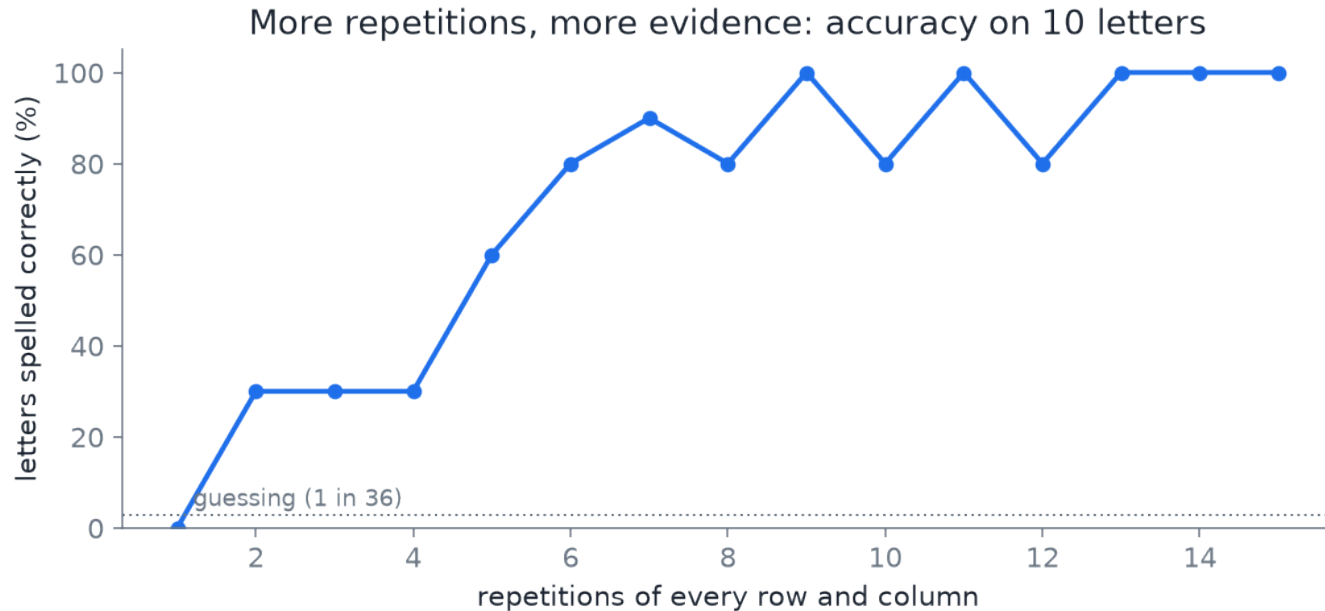
Accumulated evidence for each row and column as repetitions proceed.

```
for each flash: scores[group] += w @ x + b
at the end:    mean = scores[group] / counts[group]
best row = argmax over rows, best col = argmax over cols
letter    = matrix[best row][best col]
```

Why averaging helps

- Noise is independent between flashes; the response is not.
- Summing 12 repetitions increases the signal by 12 and the noise by $\sqrt{12}$.
- The signal-to-noise ratio therefore improves by a factor of about 3.5.
- Dividing by the number of flashes keeps groups comparable if a block ends early.

Accuracy versus number of repetitions



Letter accuracy as a function of the number of repetitions, from a recorded session.

Spelling rate

12 reps -> 22 s/letter -> ~2.7 letters/min

6 reps -> 11 s/letter -> ~5.5 letters/min

but only if accuracy holds up

Dynamic stopping

Instead of a fixed number of repetitions, flashing continues until the difference between the leading group and the others exceeds a threshold. Easy letters then take about 6 seconds and difficult ones longer.

Processing one letter, step by step

- 1 Stimulus client** draws the cue, flashes group 7, writes stimulus.rowFlash into the buffer
- 2 Buffer** stamps that event with the current sample count and stores it
- 3 Acquisition client** keeps pushing samples from the amplifier, 128 per second, knowing nothing about flashes
- 4 Signal processor** wakes on the event, waits for 77 more samples, slices, filters, scores, adds to the row total
- 5 Repeat 144 times** approximately 22 seconds in total
- 6 Signal processor** picks the best row and column, writes classifier.prediction R
- 7 Stimulus client** was blocked waiting for exactly that event — shows the letter and appends it to the text

The four programs above exchange no function calls; they interact only through the buffer.

Data storage and file formats

```
sessions/2026-09-23_1540/  
  header          channels, rate, dtype  
  header.txt      the same, readable  
  samples         raw EEG, binary  
  events          every event, in order  
  calibration_epochs.npz  
  clsfr.pkl       the trained model  
  training_summary.json
```

A recorded session is the basis for offline analysis and for reproducing a result; a recording session with a participant cannot be repeated.

1

FieldTrip offline format

The same layout written by `buffer_bci`, so existing MATLAB tools read these recordings directly.

2

Raw data first

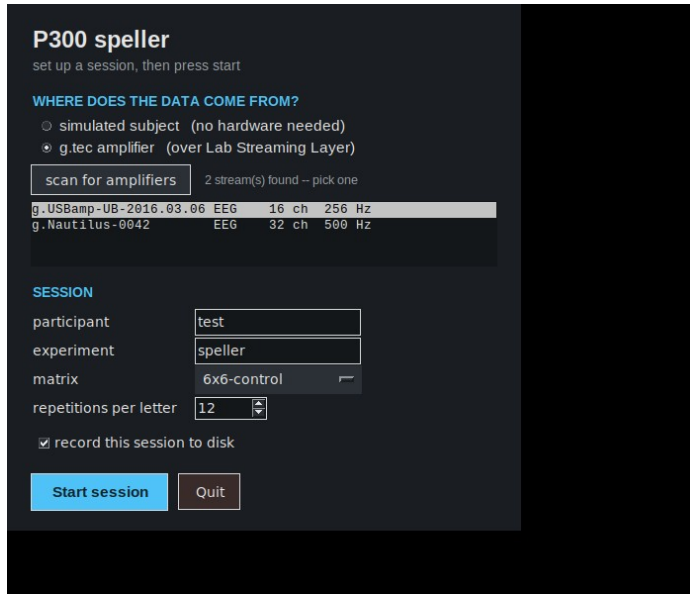
Samples and events are written as they arrive; epochs, models and summaries are derived files that can be recomputed.

3

One directory per session

A result is reproduced by running the analysis on the same directory again.

Graphical user interface



The launcher window used to start a session.

1 Implemented in tkinter

Part of the standard library, so no additional installation is required.

2 The interface only writes events

Pressing "Calibrate" writes startPhase.cmd into the buffer; it performs no other action.

3 Timing is not handled here

Stimulus timing is in a separate process, so a slow redraw cannot delay a flash.

4 Threading rule

Buffer reads run on a background thread and results are passed to the interface thread through a queue.

Testing and reproducibility

1 Simulated participant

The simulator produces an event-related response of known amplitude and latency, with interference, blinks and drift.

2 Compressible experiment time

All delays pass through one clock class with a speed factor, so a 22-second letter can be run in a fraction of a second.

3 Fixed random seeds

The same seed produces the same flash sequence and the same result, so a failing test is reproducible.

4 Protocol compatibility test

One test drives this server with the original FieldTrip client code, verifying the protocol implementation.

145 tests

40 s full run

0 devices needed

100× time compression

Demonstration

```
$ pypeller run --simulate --speed 4
```

```
# or, with the amplifier:
```

```
$ pypeller run --lsl
```

```
# and just the GUI:
```

```
$ pypeller gui
```

If errors occur

Use the failure as an exercise: identify which process is responsible and how this can be established. The buffer contains every event that has occurred.

Points to observe

- The cue, followed by 144 flashes for one letter
- The training window: AUC, ERP and discriminability map
- Feedback: the decoded letter appearing in the text line
- Stopping the signal processor during a run and restarting it
- Pausing and switching phase while a block is running

Possible extensions and project topics

1 Dynamic stopping

Stop the flashing when the leading group is sufficiently ahead. About 20 lines; reduces the time per letter.

2 Other selection tasks

Simple games such as a grid target task or tic-tac-toe. The decoder is unchanged and the target is known, so trials are labelled.

3 A different alphabet

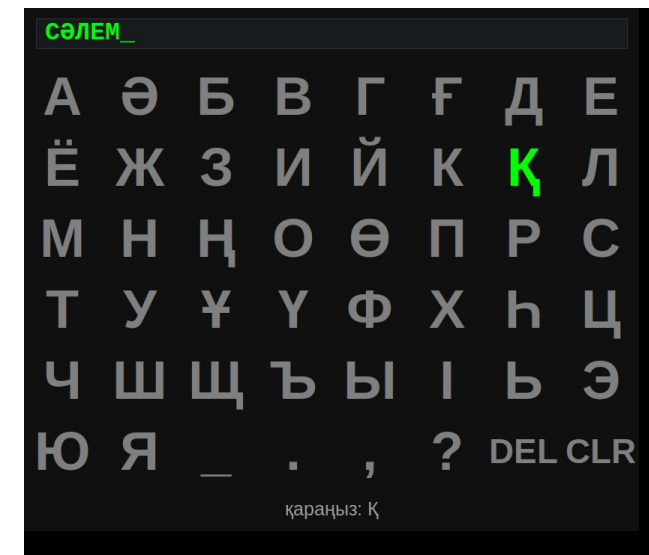
The Kazakh 6×8 layout is already implemented: 42 letters, with the interface language following the layout.

4 Artefact handling

Detect and reject blinks online rather than discarding epochs afterwards.

5 A different classifier

Any model implementing fit and decision_function can be substituted.



The same system with the Kazakh 6×8 layout.

Each of these affects a single module and can be tested with the simulator before any recording.

CONCLUSION

Summary

1 Time is measured in samples

Events are indexed by sample number rather than by clock time, which makes stimulus and EEG directly comparable.

3 Repeated weak evidence

A single flash is uninformative; 24 scores decide a letter, because signal accumulates faster than noise.

2

Components communicate through a shared buffer

Samples and events in one store; no direct calls between programs.

4

The system is testable offline

A simulated participant and compressible time allow the whole system to be tested without hardware.

the code github.com/berdakh/pyspeller
the tutorial berdakh.github.io/pyspeller

*Reading before the laboratory session:
the tutorial pages on the architecture and
the first run.*